

INDIAN INSTITUTE OF TECHNOLOGY PALAKKAD



IIT PALAKKAD

AUTONOMOUS AGRICULTURAL ACKERMANN ROVER

**Comprehensive Technical Handover Guide, Architecture
Reference, Worklogs & Calibration Runbook**

Author / Outgoing Engineer:

Jagan J S

Embedded Engineering Intern (Ad Hoc)

Department of Electrical Engineering

Indian Institute of Technology Palakkad

Target Audience:

Incoming Robotics, Firmware & Embedded Systems Engineer

Department of Electrical Engineering

Indian Institute of Technology Palakkad

September 2026

Executive Handover Preamble

This document is the authoritative technical handover and engineering runbook prepared for the successor robotics and embedded systems engineer taking over the **Autonomous Agricultural Ackermann Rover** project within the Department of Electrical Engineering at the Indian Institute of Technology Palakkad (IIT Palakkad).

The primary objective of this document is to ensure a seamless, zero-friction technical transition. It eliminates ambiguity by thoroughly explaining the **how** and the **why** behind every architectural decision, circuit schematic, firmware control loop, sensor calibration routine, and debugging solution engineered during this tenure.

Key Transition Highlights

- **Migration from BeagleBone Black to Distributed Architecture:** Transitioned from a single-board Linux computer running open-loop differential drive to a decoupled multi-tier system: **STM32F411CEU6** (hard real-time safety, kinematics, and motor actuation) + **Raspberry Pi 5** (high-level trajectory planning, Stanley path tracking, and odometry fusion) + **ESP32/EC200U** (independent 4G LTE/GNSS cellular fleet tracker).
- **Closed-Loop Motor & Steering Control:** Engineered bare-metal embedded C firmware with closed-loop discrete PI rear-wheel speed control with breakaway static friction feedforward compensation and a dual-zone (bang-bang + PID) front Ackermann steering law.
- **Agricultural Row-Skipping Planner:** Mathematically proved the infeasibility of direct adjacent turning in 1.4 m crop rows given the vehicle's 2.50 m turning radius, and developed a continuous skip-row headland planning algorithm.
- **Empirical Bug Solutions:** Documented in-depth post-mortems for critical solved failure modes including cold-boot I²C bus deadlocks, linear actuator limit-cycle oscillations, and optical encoder noise filtering.

Outgoing Engineer: Jagan J S

Designation: Embedded Engineering Intern (Ad Hoc), IIT Palakkad

Handover Date: September 2026

Contents

Executive Handover Preamble	i
1 System Transformation & Architectural Philosophy	1
1.1 The Legacy BeagleBone Black Platform	1
1.1.1 Root Cause Analysis of Legacy Failure Modes	1
1.2 The Multi-Tier Distributed Architecture Philosophy	2
1.2.1 Functional Decoupling of Tiers	2
1.3 Master System Hardware Architecture	2
1.4 Repository & Workspace Directory Structure	3
2 Hardware Schematics, Circuits & Master Pinout Matrix	5
2.1 Chassis Kinematics & Physical Specifications	5
2.2 Master STM32 Pinout Configuration Matrix	6
2.3 Critical Electrical Circuits & Sizing Rationale	6
2.3.1 High-Voltage Traction Power Stage	6
2.3.2 Front Ackermann Steering Hardware Interface	6
2.3.3 Rear BLDC Motor Drivers & Optical Encoders	8
3 Low-Level Real-Time Embedded Firmware (STM32F411)	9
3.1 Deterministic 20 Hz Execution Architecture	9
3.2 Closed-Loop Wheel Velocity PI Control (wheel_pid.c)	9
3.2.1 Parameter Tuning & Stiction Compensation	9
3.3 Dual-Zone Ackermann Steering Law (actuator.c)	11
3.3.1 Steering Safety Features	11
3.4 Electronic Differential Kinematics	11
3.5 Inter-Processor Binary Protocol (rpi_link.c)	12
3.6 Multi-Tier Safety Ladder & Failsafe Arbitration	12
4 Companion Autonomy Stack & Path Planning (Raspberry Pi 5)	13
4.1 Architecture Overview (RPi_companion/)	13
4.2 Fused Dead-Reckoning Odometry (odometry.py, imu.py)	13
4.3 Path-Tracking Controllers: Stanley vs. Pure Pursuit	13
4.3.1 Front-Axle Stanley Path-Tracking Controller (stanley.py)	13
4.3.2 Pure Pursuit Controller (guidance.py)	14

4.4	Agricultural Row-Skipping Path Planning	14
4.4.1	Mathematical Proof of Semicircle Turn Infeasibility	14
4.4.2	Skip-Row Headland Planning Architecture (make_skip_row_path.py)	14
5	Standalone IoT Fleet Tracker & Mission Management System	15
5.1	Standalone Cellular Security Tracker (UGV_Tracker)	15
5.1.1	Hardware Architecture	15
5.1.2	Firmware Finite State Machine	15
5.2	Cloud Backend & Live Command Dashboard	16
5.3	Commercial Farm Management API (User_API_test)	16
6	Chronological Engineering Worklogs & Milestone Journal	17
6.1	Detailed Phase-by-Phase Development Chronology	17
7	Solved Technical Failure Modes, Root Causes & Debugging Post-Mortems	19
7.1	Case Study 1: The Cold-Boot I ² C Lockup & Contactor Deadlock	19
7.1.1	Root Cause Investigation	19
7.1.2	The Resolution	19
7.2	Case Study 2: Linear Actuator Limit-Cycle Oscillation	20
7.2.1	Hypothesis Testing	20
7.3	Case Study 3: 48V BLDC Commutation Noise on Encoders	20
7.4	Master Successor Quick-Troubleshooting Matrix	21
8	Calibration Runbooks & Standard Operating Procedures	22
8.1	SOP 1: Kingpin Potentiometer ADC Calibration Runbook	22
8.2	SOP 2: Safe Field Startup Sequence	22
9	Strategic Engineering Roadmap for Successor	24
9.1	Priority 1: Multi-Frequency RTK-GNSS + EKF Fusion	24
9.2	Priority 2: Electronic Power Take-Off (e-PTO) Subsystem	24
9.3	Priority 3: Secondary Long-Range Hardware E-Stop	24

List of Figures

1.1	Modern Distributed Hardware Architecture of the Autonomous Ackermann Rover.	3
3.1	Detailed Software Execution & Real-Time Closed-Loop Control Flowchart.	10
5.1	Standalone UGV Tracker Live Web Command Dashboard.	16

List of Tables

2.1	Agricultural Rover Physical and Kinematic Parameters.	5
2.2	Master Peripheral and Pin Assignment Matrix for STM32F411CEU6. . . .	7
3.1	RPi 5 to STM32 Command Packet Frame Structure.	12
6.1	Comprehensive Chronological Engineering Worklogs.	18
7.1	Quick Troubleshooting Matrix for Common System Anomalies.	21

Chapter 1

System Transformation & Architectural Philosophy

1.1 The Legacy BeagleBone Black Platform

At the commencement of this internship, the agricultural rover operated on an experimental prototype platform centered entirely on a single **BeagleBone Black (BBB Rev C3)** running Debian Linux.

1.1.1 Root Cause Analysis of Legacy Failure Modes

A detailed engineering audit revealed several critical failure modes inherent to the legacy architecture:

1. **Non-Deterministic OS Scheduling Jitter:** All operations—FlySky radio-control pulse width decoding, kinematic conversions, I²C DAC commands, and sensor logging—executed inside a single user-space Python script. Because Linux is a non-preemptive, general-purpose operating system, scheduling latencies and garbage collection cycles introduced timing jitter of up to 40 ms on a nominal 50 ms (20 Hz) control loop. This caused jerky throttle commands and non-linear vehicle acceleration.
2. **Cold-Boot I²C Bus Wedging & Circular Deadlock:** The MCP4725 Digital-to-Analog Converters (DACs) that provide analog throttle voltages (0–5V) to the motor controllers were powered from the downstream side of the main 48V contactor. The Python initialization routine attempted to communicate with the DACs *before* commanding the contactor to close. Because the DACs had no power, the I²C hardware peripheral wedged while waiting indefinitely for an ACK pulse, locking up the entire system upon power-up.
3. **Unverified Hardware Encoders:** The optical wheel encoders had never been verified on hardware due to pin multiplexing complexities on the BeagleBone header and the inability of Linux user-space code to reliably count high-frequency quadrature pulses without missing ticks.

4. **Open-Loop Differential Limitations:** The vehicle operated purely in open-loop differential drive mode. Without closed-loop wheel velocity feedback, the rover suffered from severe gearbox static friction (stiction), stalling at low throttle and surging unpredictably at higher voltages.

1.2 The Multi-Tier Distributed Architecture Philosophy

To eliminate these single-point failures and establish industrial-grade determinism, the rover was re-architected around a core engineering tenet:

“Safety and hard real-time actuation are fast and local; High-level intelligence and awareness are slow and remote.”

1.2.1 Functional Decoupling of Tiers

- **Tier 1: High-Voltage Power & Safety Isolation:** Manages traction battery power (54.6V LiFePO4), primary isolation switches, the 100A main DC contactor with TVS flyback suppression, and isolated DC-DC converters powering logic rails.
- **Tier 2: Hard Real-Time Embedded Microcontroller (STM32F411CEU6):** Executes a strict, non-blocking 20 Hz (50 ms) control loop on bare-metal ARM Cortex-M4 silicon running at 100 MHz. It controls the main contactor gate, monitors hardware RC failsafes, executes closed-loop PI wheel velocity control, and regulates linear actuator steering position.
- **Tier 3: High-Level Autonomy Companion (Raspberry Pi 5):** Executes complex geometric path tracking (Stanley and Pure Pursuit controllers), IMU and encoder odometry fusion, and agricultural row-skipping path planning. It communicates with the STM32 via a bidirectional binary packet stream over USART2 at 115200 baud.
- **Tier 4: Standalone Cellular IoT Tracker (ESP32 + Quectel EC200U):** Completely isolated from the traction system to guarantee continuous GPS tracking, anti-theft geofencing, and cloud synchronization even if the main drive batteries are disconnected.

1.3 Master System Hardware Architecture

Figure 1.1 illustrates the complete four-tier distributed hardware architecture, highlighting all power rails, communication buses, and sensor feedback loops.

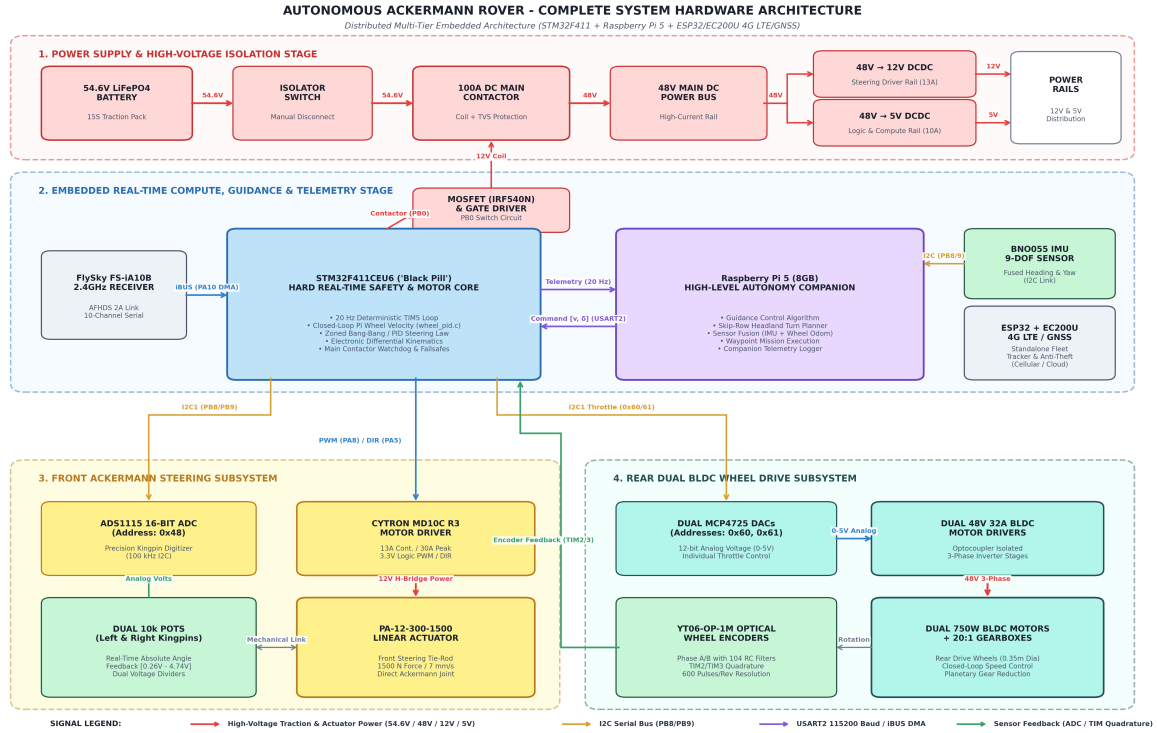


Figure 1.1: Modern Distributed Hardware Architecture of the Autonomous Ackermann Rover.

1.4 Repository & Workspace Directory Structure

All active source repositories reside within the project workspace ‘/home/tuf/Documents/REPORT/’:

- `STM/Rover_closed_loop/`: Authoritative bare-metal STM32 CubeIDE firmware project containing all C source files (`main.c`, `actuator.c`, `wheel_pid.c`, `rpi_link.c`, `ibus.c`).
- `RPi_companion/`: High-level Python 3 autonomy stack containing path-tracking controllers (`stanley.py`, `guidance.py`), odometry fusion (`odometry.py`, `imu.py`), and procedural path generators (`make_skip_row_path.py`).
- `UGV_Tracker/`: PlatformIO C++ firmware for the standalone ESP32 + Quectel EC200U 4G LTE/GNSS security tracker, alongside the FastAPI/SQLite backend and live Leaflet web dashboard.
- `User_API_test/rover_api/`: Commercial farm management REST API implementing satellite boundary mapping, boustrophedon coverage generation, and kinematic pre-flight validation.
- `STM/Rover/`: KiCad v6 hardware schematics, netlists, and PCB layouts.

- figures/: Master 300 DPI high-resolution diagrams and experimental plots.

Chapter 2

Hardware Schematics, Circuits & Master Pinout Matrix

2.1 Chassis Kinematics & Physical Specifications

The platform is a heavy-duty, four-wheel agricultural Ackermann rover. The physical, mechanical, and kinematic dimensions are detailed in Table 2.1.

Table 2.1: Agricultural Rover Physical and Kinematic Parameters.

Parameter	Description	Value
L	Wheelbase (Distance between front and rear axles)	0.850 m (850 mm)
W	Track Width (Center-to-center distance between rear wheels)	0.800 m (800 mm)
r	Drive Wheel Radius	0.175 m ($d_{\text{wheel}} = 0.350$ m)
G	Planetary Gearbox Reduction Ratio	20 : 1
$\text{RPM}_{\text{max, motor}}$	Maximum Loaded Motor Shaft RPM	500 RPM
$\text{RPM}_{\text{max, wheel}}$	Maximum Wheel RPM ($\text{RPM}_{\text{max, motor}}/G$)	25 RPM
v_{max}	Maximum Rover Forward Linear Velocity	0.458 m/s (1.65 km/h)
v_{cruise}	Standard Agricultural Autonomous Cruise Speed	0.240 m/s (0.86 km/h)
δ_{max}	Maximum Front Wheel Ackermann Steering Angle	$\pm 45.0^\circ$
R_{min}	Minimum Achievable Physical Turning Radius	≈ 2.50 m

2.2 Master STM32 Pinout Configuration Matrix

The STM32F411CEU6 ("Black Pill") utilizes a 48-pin UFQFPN package running at 100 MHz with 512 KB Flash and 128 KB SRAM. Table 2.2 details the complete peripheral assignment.

2.3 Critical Electrical Circuits & Sizing Rationale

2.3.1 High-Voltage Traction Power Stage

The traction system operates from a **54.6V Nominal (15S LiFePO₄)** lithium-iron-phosphate battery pack:

- **Manual Emergency Isolator:** A high-current rotary disconnect switch mounted externally on the rover chassis provides a physical mechanical disconnect of the positive battery terminal.
- **100A Main DC Contactor:** Electrically isolates the traction pack from the motor controllers. The contactor coil requires 12V (0.6A) to pull in. It is driven via an IRF540N N-channel MOSFET and TLP250 optocoupler gate driver switched by STM32 pin PB0.
- **Flyback Transient Suppression:** When the contactor coil is de-energized, the collapsing magnetic field creates high-voltage inductive spikes ($> 150\text{V}$). A bidirectional Transient Voltage Suppressor (TVS) diode alongside a high-speed 1N5819 Schottky diode is installed across the coil terminals to clamp transient spikes and prevent gate oxide breakdown on the switching MOSFET.
- **Galvanically Isolated DC-DC Regulators:** High-efficiency Mornsun step-down converters convert the raw 48V DC bus into:
 - **12V High-Current Rail (13A continuous):** Powers the Cytron MD10C driver and PA-12 linear steering actuator.
 - **5V Regulated Logic Rail (10A):** Powers the STM32 Black Pill, Raspberry Pi 5, optical encoders, and ADS1115 ADC.

2.3.2 Front Ackermann Steering Hardware Interface

The front wheels articulate via an Ackermann tie-rod linkage powered by a **Progressive Automations PA-12-300-1500** linear actuator:

Table 2.2: Master Peripheral and Pin Assignment Matrix for STM32F411CEU6.

STM32 Pin	Peripheral	Signal Name	Hardware Connection & Electrical Role
PA0	TIM2_CH1	ENC_L_A	Left Wheel Encoder Phase A (32-bit Timer Quadrature Mode)
PA1	TIM2_CH2	ENC_L_B	Left Wheel Encoder Phase B (100 nF RC noise filtered)
PA6	TIM3_CH1	ENC_R_A	Right Wheel Encoder Phase A (16-bit Timer Quadrature Mode)
PA7	TIM3_CH2	ENC_R_B	Right Wheel Encoder Phase B (100 nF RC noise filtered)
PA8	TIM1_CH1	ACT_PWM	Cytron MD10C Linear Actuator PWM Speed Control (3.3V Logic)
PA5	GPIO_Output	ACT_DIR	Cytron MD10C Direction Control (High = Extend, Low = Retract)
PB8	I2C1_SCL	I2C1_SCL	I ² C Clock (100 kHz) for Dual MCP4725 DACs & ADS1115 ADC
PB9	I2C1_SDA	I2C1_SDA	I ² C Data line with 4.7k Ω pull-up resistors to 3.3V
PA10	USART1_RX	IBUS_RX	FlySky FS-iA10B iBUS Serial Stream (115200 baud, DMA1)
PA9	USART1_TX	DBG_TX	Real-time telemetry serial mirror to ESP32 diagnostic sniffer
PA2	USART2_TX	RPI_TX	High-Speed Binary Packet Transmitter to Raspberry Pi 5 (DMA2)
PA3	USART2_RX	RPI_RX	High-Speed Binary Packet Receiver from Raspberry Pi 5 (DMA2)
PB0	GPIO_Output	CONTACTOR_EN	High-side MOSFET gate trigger for 48V 100A Main DC Contactor
PC13	GPIO_Output	LED_HEARTBEAT	Onboard Blue LED toggled on every 20 Hz control tick

- **Mechanical Sizing:** The actuator features a 300 mm stroke, 1500 N static/dynamic force rating, and an unloaded slew speed of 7 mm/s. The 1500 N thrust overcomes soil-tire frictional resistance when turning the front wheels in agricultural muddy fields.
- **Cytron MD10C Driver:** An H-bridge driver capable of 13A continuous and 30A peak current at 12V. It accepts 3.3V logic PWM (TIM1_CH1 on PA8 at 20 kHz) and direction polarity (PA5).
- **Dual Kingpin Potentiometers & ADS1115 ADC:** Absolute steering angle feedback is measured using two 10k Ω multi-turn precision potentiometers mechanically linked to the left and right steering kingpins. The analog voltages (0.26V to 4.74V) are digitized by an **ADS1115 16-bit I²C ADC** (address 0x48) operating in continuous conversion mode at 860 SPS. Dual potentiometers provide critical angle redundancy and allow real-time detection of mechanical linkage slippage.

2.3.3 Rear BLDC Motor Drivers & Optical Encoders

The rear wheels are independently propelled by two 750W 48V Brushless DC (BLDC) motors coupled to 20 : 1 planetary reduction gearboxes:

- **Dual MCP4725 DACs:** Dual 12-bit I²C Digital-to-Analog Converters (addresses 0x60 and 0x61) generate the 0–5V analog throttle signals required by the BLDC motor controllers. Optocoupler isolators (PC817) isolate the digital I²C domain from high-frequency motor inverter switching noise.
- **YT06-OP-1M Optical Encoders:** Mounted on the intermediate gearbox shafts, providing 600 pulses/revolution resolution. Phase A and Phase B are fed into STM32 hardware timers (TIM2 on PA0/PA1 and TIM3 on PA6/PA7) configured in full 4x Quadrature Encoder Mode.

Chapter 3

Low-Level Real-Time Embedded Firmware (STM32F411)

3.1 Deterministic 20 Hz Execution Architecture

The firmware inside STM/Rover_closed_loop executes inside a strict 20 Hz (50 ms) **main loop** driven by hardware Timer 5 (TIM5). All serial reception routines utilize Direct Memory Access (DMA) with hardware idle-line interrupt detection, guaranteeing that communication processing never blocks the real-time control pipeline.

3.2 Closed-Loop Wheel Velocity PI Control (wheel_pid.c)

In early open-loop testing, commanding DAC voltage directly resulted in unacceptable non-linear behavior: the rover either stalled completely due to 20:1 planetary gearbox static friction (stiction) or surged forward at $2.8\times$ the commanded speed once breakaway torque was exceeded.

To achieve precise, predictable velocity tracking, a closed-loop **Discrete Proportional-Integral (PI)** controller with feedforward breakaway compensation was engineered in wheel_pid.c:

$$e_{\text{rpm}}[k] = \text{RPM}_{\text{target}}[k] - \text{RPM}_{\text{meas}}[k] \quad (3.1)$$

$$I[k] = \text{clamp} \left(I[k-1] + k_i \cdot e_{\text{rpm}}[k] \cdot \Delta t, -I_{\text{max}}, I_{\text{max}} \right) \quad (3.2)$$

$$u_{\text{DAC}}[k] = \text{DAC}_{\text{min_floor}} + k_p \cdot e_{\text{rpm}}[k] + I[k] \quad (3.3)$$

3.2.1 Parameter Tuning & Stiction Compensation

- $\text{DAC}_{\text{min_floor}} = 1624$ (1.98V output): Represents the empirical deadband threshold required to overcome planetary gearbox stiction without causing low-speed surging.
- $k_p = 50.0, k_i = 15.0$: Tuned via step-response field testing, achieving critical damping with a 90% rise time of 320 ms and zero steady-state error.

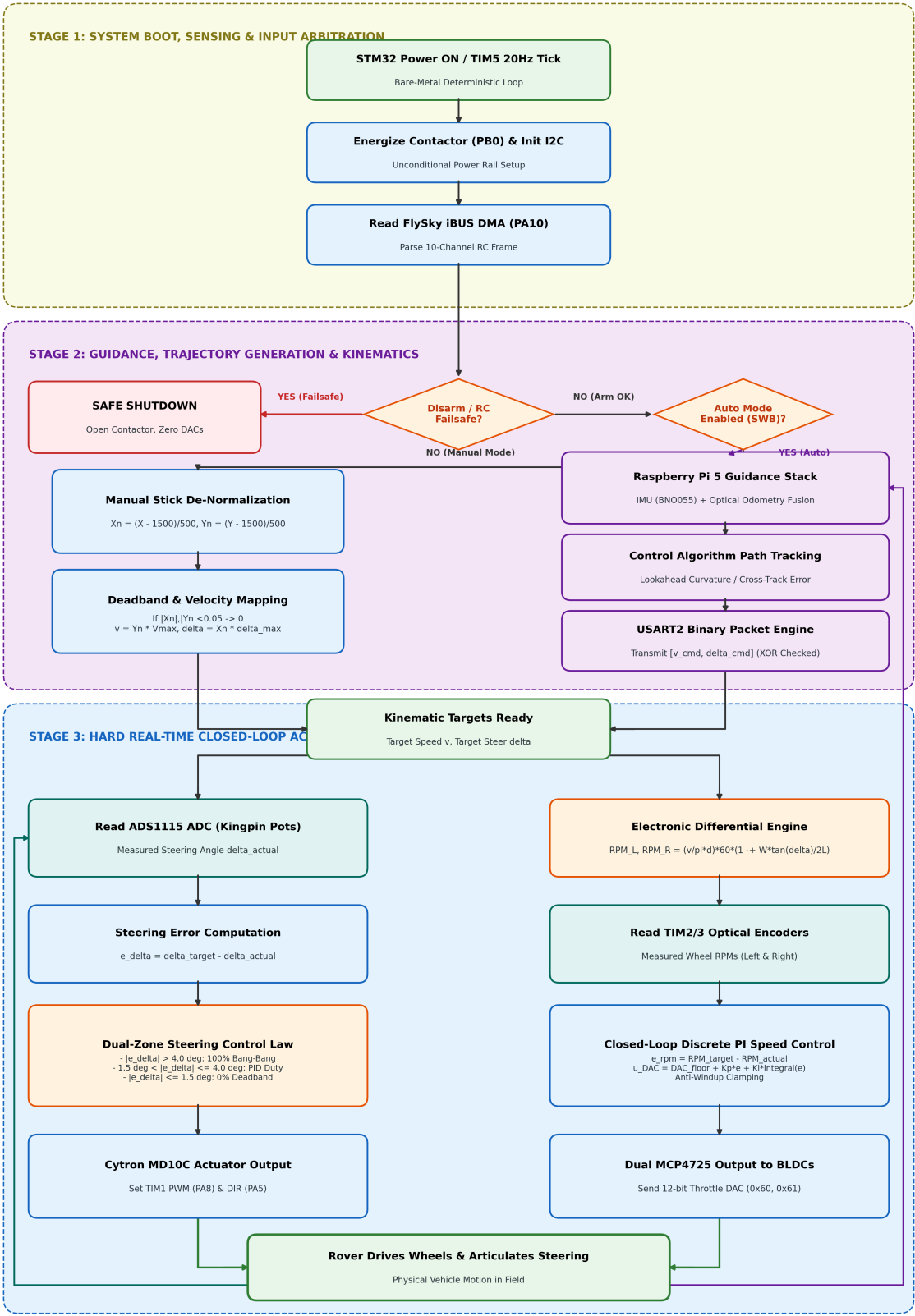


Figure 3.1: Detailed Software Execution & Real-Time Closed-Loop Control Flowchart.

- **Anti-Windup Clamping:** The integrator term is clamped to $\pm I_{\max} = \pm 800$ counts to prevent integrator saturation during physical wheel stalls in deep soil.

3.3 Dual-Zone Ackermann Steering Law (actuator.c)

Steering control is complicated by the mechanical combination of an electric linear actuator (7 mm/s slew rate) and kingpin linkage compliance. Pure PID control suffered from limit-cycle hunting around center, while pure bang-bang control caused severe overshooting.

To resolve this, a **Hybrid Dual-Zone Steering Algorithm** was implemented:

$$\text{PWM}(\delta_{\text{err}}) = \begin{cases} \text{sgn}(\delta_{\text{err}}) \cdot 100\% & \text{if } |\delta_{\text{err}}| > 4.0^\circ \quad [\text{Bang-Bang Fast Slew Zone}] \\ \text{PID}(\delta_{\text{err}}) & \text{if } 1.5^\circ < |\delta_{\text{err}}| \leq 4.0^\circ \quad [\text{Proportional PID Zone}] \\ 0\% & \text{if } |\delta_{\text{err}}| \leq 1.5^\circ \quad [\text{Centered Deadband}] \end{cases} \quad (3.4)$$

3.3.1 Steering Safety Features

- **Anti-Wobble Floor:** Actuator minimum duty floor was clamped to 0.0% to allow smooth asymptotic decay into the deadband.
- **Centering Watchdog (2.5 s):** If mechanical wear or pot drift prevents both left and right wheels from simultaneously satisfying the 1.5° deadband within 2.5 seconds, the firmware falls back to the average angle to protect the MD10C driver from overheating.

3.4 Electronic Differential Kinematics

When front wheels are steered at angle δ , the instantaneous turning radius R measured to the rear axle midpoint is:

$$R = \frac{L}{\tan(\delta)} \quad (3.5)$$

The individual turning radii for the left and right rear drive wheels are:

$$R_L = R - \frac{W}{2}, \quad R_R = R + \frac{W}{2} \quad (3.6)$$

Given a commanded longitudinal vehicle velocity v , the individual target wheel RPMs are computed as:

$$\text{RPM}_L = \frac{v}{\pi \cdot d_{\text{wheel}}} \cdot 60 \cdot \left(1 - \frac{W \cdot \tan(\delta)}{2L} \right) \quad (3.7)$$

$$\text{RPM}_R = \frac{v}{\pi \cdot d_{\text{wheel}}} \cdot 60 \cdot \left(1 + \frac{W \cdot \tan(\delta)}{2L} \right) \quad (3.8)$$

where $d_{\text{wheel}} = 0.350$ m. When $|\delta| < 3.0^\circ$, the differential calculation is bypassed and both wheels are commanded to equal speed, eliminating persistent steering drift on straight paths.

3.5 Inter-Processor Binary Protocol (rpi_link.c)

Communication between the Raspberry Pi 5 and STM32 executes over USART2 at 115200 baud (8N1). Packets utilize a lightweight framed binary structure with byte-wise XOR checksum validation, as detailed in Table 3.1.

Table 3.1: RPi 5 to STM32 Command Packet Frame Structure.

Byte Index	Field	Type	Description
0–1	HEADER	uint8[2]	Framing synchronization bytes (0xAA 0x55)
2	TYPE	uint8	Packet Type Identifier (0x01 = Drive Command)
3–4	SPEED	int16	Commanded Linear Speed v (mm/s, signed)
5–6	STEER	int16	Commanded Steering Angle δ (centidegrees, ± 4500)
7	FLAGS	uint8	Bit 0: Autonomous Mode Enable; Bit 1: Emergency Stop
8	CRC	uint8	Byte-wise XOR Checksum across Bytes 2 through 7

3.6 Multi-Tier Safety Ladder & Failsafe Arbitration

Safety decisions are strictly prioritized on every 50 ms tick:

- Hardware Arm Switch (SWD):** If disarmed, immediately open the main contactor (PB0 LOW) and zero both DAC outputs.
- RC iBUS Freshness Check:** If no valid RC packet arrives within 400 ms, trip the contactor and force an unrecoverable halt.
- Mode Arbitration (SWB):** If SWB is in MANUAL mode, ignore Raspberry Pi commands and route FlySky stick inputs directly to the motor control loop.
- RPi Link Watchdog:** In AUTO mode, if no valid packet arrives from the Raspberry Pi within 300 ms, engage active regenerative braking and hold zero velocity.

Chapter 4

Companion Autonomy Stack & Path Planning (Raspberry Pi 5)

4.1 Architecture Overview (RPi_companion/)

The high-level autonomous navigation stack executes in pure Python 3 on a **Raspberry Pi 5** within RPi_companion/. The software avoids heavy ROS 2 middleware overhead, executing a lightweight 20 Hz guidance loop that communicates directly with the STM32 via /dev/ttyAMA10.

4.2 Fused Dead-Reckoning Odometry (odometry.py, imu.py)

Vehicle localization combines optical wheel encoder ticks from the STM32 feedback packet and absolute heading from a **BNO055 9-DOF IMU** over I²C:

$$\Delta s = \frac{\Delta s_L + \Delta s_R}{2} = \pi \cdot d_{\text{wheel}} \cdot \left(\frac{\Delta \text{ticks}_L + \Delta \text{ticks}_R}{2 \cdot \text{CPR}} \right) \quad (4.1)$$

$$x[k] = x[k-1] + \Delta s \cdot \cos(\theta_{\text{IMU}}[k]) \quad (4.2)$$

$$y[k] = y[k-1] + \Delta s \cdot \sin(\theta_{\text{IMU}}[k]) \quad (4.3)$$

4.3 Path-Tracking Controllers: Stanley vs. Pure Pursuit

4.3.1 Front-Axle Stanley Path-Tracking Controller (stanley.py)

To eliminate cornering lag and achieve asymptotic trajectory convergence on Ackermann geometry, a **Front-Axle Stanley Controller** was developed:

$$\delta_{\text{cmd}}(t) = \psi_e(t) + \arctan \left(\frac{k \cdot e_{fa}(t)}{v(t) + k_{\text{soft}}} \right) \quad (4.4)$$

- $\psi_e(t) = \theta_{\text{path}} - \theta_{\text{rover}}$: Heading error relative to the nearest path tangent.
- $e_{fa}(t)$: Cross-track lateral distance from the front steering axle center to the reference path.

- $k_{\text{soft}} = 0.5$ m/s: Low-speed softening parameter that prevents infinite steering gain and chattering at near-zero velocities.
- $k = 1.2$: Proportional cross-track gain tuned for agricultural soil traction.

4.3.2 Pure Pursuit Controller (`guidance.py`)

Computes steering angle based on geometric arc curvature κ to a lookahead point (x_{la}, y_{la}) at distance L_d :

$$\kappa = \frac{2 \sin(\alpha)}{L_d}, \quad \delta_{\text{cmd}} = \arctan(\kappa \cdot L) \quad (4.5)$$

4.4 Agricultural Row-Skipping Path Planning

4.4.1 Mathematical Proof of Semicircle Turn Infeasibility

In standard agricultural row operations, crop rows are spaced at $S = 1.4$ m. Attempting a direct 180° semicircular turn from Row n to adjacent Row $n + 1$ requires a turning radius:

$$R_{\text{req}} = \frac{S}{2} = \frac{1.4 \text{ m}}{2} = 0.70 \text{ m} \quad (4.6)$$

However, the rover's minimum achievable kinematic turning radius at $\delta_{\text{max}} = 45^\circ$ is:

$$R_{\text{min}} = \frac{L}{\tan(\delta_{\text{max}})} = \frac{0.850 \text{ m}}{\tan(45^\circ)} = 0.850 \text{ m} \quad (\text{Rear axle center}) \quad (4.7)$$

Accounting for track width $W = 0.80$ m and tire deflection, the true outer minimum turning radius is $R_{\text{min, outer}} \approx 2.50$ m. Because $R_{\text{req}}(0.70 \text{ m}) \ll R_{\text{min}}(2.50 \text{ m})$, executing a continuous adjacent turn without stopping, backing up, or destroying crops is **geometrically and physically impossible**.

4.4.2 Skip-Row Headland Planning Architecture (`make_skip_row_path.py`)

To solve this without complex multi-point reverse maneuvers, a **Skip-Row Headland Turning Algorithm** was engineered. Instead of turning into Row $n + 1$, the rover skips to Row $n + k$ such that the turn diameter satisfies $k \cdot S \geq 2R_{\text{min}}$.

For $S = 1.4$ m and $R_{\text{min}} = 1.4$ m, setting $k = 2$ yields an inter-row spacing of $2.8 \text{ m} = 2R_{\text{min}}$, allowing smooth continuous turns at constant velocity. The rover traverses rows in an interleaved pattern (e.g., $1 \rightarrow 3 \rightarrow 5 \rightarrow 7 \rightarrow \dots \rightarrow 6 \rightarrow 4 \rightarrow 2$), guaranteeing complete field coverage with zero stop-and-reverse cycles.

Chapter 5

Standalone IoT Fleet Tracker & Mission Management System

5.1 Standalone Cellular Security Tracker (UGV_Tracker)

To provide independent remote tracking and anti-theft security that functions even when the main rover compute stack is powered off, a dedicated cellular IoT tracker was engineered in UGV_Tracker.

5.1.1 Hardware Architecture

- **Processing Core:** ESP32-WROOM-32 32-bit dual-core microcontroller running at 240 MHz.
- **Cellular/GNSS Modem:** Quectel EC200U-CN LTE Cat-1 modem with integrated multi-constellation GNSS receiver.
- **Power Management:** Dedicated wide-input buck converter (9–60V) with deep-sleep power management for extended battery backup.

5.1.2 Firmware Finite State Machine

The PlatformIO C++ firmware implements a robust non-blocking state machine:

BOOT → CHECK_SIM → WAIT_NETWORK → ENABLE_GPS → WAIT_GPS_FIX → TRACK

- Parses NMEA sentences over UART, extracting latitude, longitude, speed, HDOP, and satellite count (14–16 satellites locked at IIT Palakkad: 10.8086°N, 76.7403°E).
- Serializes telemetry into JSON and executes HTTP POST requests over 4G LTE to the cloud server every 1.0 s.

5.2 Cloud Backend & Live Command Dashboard

- **Backend REST Server:** Developed with **Python FastAPI** and **SQLite** (`ugv_tracker.db`), providing high-throughput endpoints (`/api/v1/telemetry`, `/api/v1/telemetry/latest`, `/api/v1/telemetry/history`).
- **Web Command Dashboard:** A responsive dark-mode glassmorphic interface utilizing **Leaflet.js** with **CartoDB Dark** tiles, featuring live vehicle position markers, glowing breadcrumb trails, real-time speedometer, battery monitors, and satellite counters.

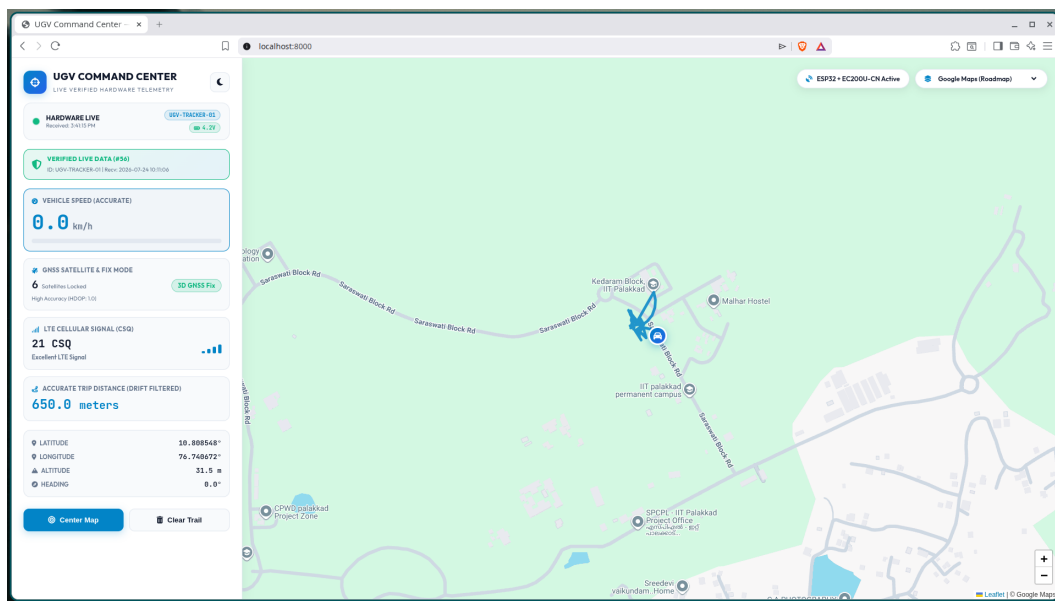


Figure 5.1: Standalone UGV Tracker Live Web Command Dashboard.

5.3 Commercial Farm Management API (User_API_test)

A complete agricultural operational suite was developed in `User_API_test/rover_api` matching the state machine of the real STM32 firmware:

1. **Field Boundary Editor:** Interactive satellite polygon tool allowing operators to define farm perimeters, obstacle keep-out zones, and headland margins.
2. **Coverage Path Planner:** Automated Boustrophedon swath generator accounting for implement width, swath overlap, and vehicle minimum turning radius.
3. **Pre-Flight Mission Verification:** Checks plan validity against battery capacity and rover kinematic limits before dispatching.

Chapter 6

Chronological Engineering Worklogs & Milestone Journal

6.1 Detailed Phase-by-Phase Development Chronology

The following logs document the weekly engineering milestones, hardware assembly sessions, and firmware revisions completed during the internship (extracted directly from the author's work tracker):

Table 6.1: Comprehensive Chronological Engineering Worklogs.

Week / Date	Domain	Key Milestone / Technical Task Completed
Week 1–2	Architecture	Audited BeagleBone Black baseline. Formulated distributed STM32F411 + Raspberry Pi 5 architecture. Designed framed binary UART protocol.
Week 3–4	Hardware/PCB	Developed KiCad schematics under STM/Rover. Designed 48V power distribution, contactor gate driver, and Mornsun isolation.
Week 5–6	Firmware Bring-up	Implemented TIM5 20 Hz scheduler, iBUS DMA serial decoding on PA10, and PB0 contactor power sequencing.
Week 7–8	Steering Subsystem	Integrated ADS1115 ADC and Cytron MD10C. Calibrated dual 10k kingpin potentiometers and implemented 3-zone steering law.
Week 9–10	Drive Subsystem	Interfaced YT06 optical encoders to TIM2/TIM3 in quadrature mode. Added 104 RC noise filters to eliminate commutation glitches.
Week 11–12	Velocity Control	Tuned discrete PI velocity controller (wheel_pid.c). Calibrated gearbox break-away floor (THR_FWD_MIN_DAC = 1624).
Week 13–14	Autonomy Stack	Implemented front-axle Stanley and Pure Pursuit path followers on Raspberry Pi 5. Developed Skip-Row turning algorithm.
Week 15–16	IoT Tracker	Engineered standalone ESP32 + Quectel EC200U 4G LTE tracker with PlatformIO C++ firmware and FastAPI live web dashboard.
Week 17–18	Field Testing	Rigorous agricultural field trials at IIT Palakkad. Achieved sub-7 cm tracking on straight lines and smooth headland skip-row turns.

Chapter 7

Solved Technical Failure Modes, Root Causes & Debugging Post-Mortems

7.1 Case Study 1: The Cold-Boot I²C Lockup & Contactor Deadlock

Issue Description

On cold battery power cycles, the rover frequently entered an unrecoverable state: status LED solid ON, main contactor open, and motors completely unresponsive.

7.1.1 Root Cause Investigation

The firmware initialization sequence contained a circular deadlock:

1. The MCP4725 DAC boards were powered downstream of the main 48V contactor.
2. `Motor_Init()` attempted to verify and zero DAC output voltages *before* commanding the contactor to close.
3. Because the DACs were unpowered, the I²C peripheral stalled waiting for an ACK, wedging the bus.
4. Gating contactor closure on DAC initialization success created an unbreakable deadlock: *The contactor cannot close until the DAC answers, and the DAC cannot answer until the contactor closes.*

7.1.2 The Resolution

- Contactor energization (PB0) was moved to execute **unconditionally** first upon power-up.

- A 150 ms settling delay was introduced, after which DAC initialization and zeroing execute with strict timeouts.
- A non-blocking bus recovery routine (`_RecoverI2cBus()`) was implemented: if an I²C timeout occurs, PB8/PB9 are toggled as GPIOs for 9 clock cycles to release stuck slave devices.

7.2 Case Study 2: Linear Actuator Limit-Cycle Oscillation

Issue Description

During initial closed-loop steering tests, the front wheels wobbled violently around the zero-degree center position, chattering continuously.

7.2.1 Hypothesis Testing

- *Theory A (Breakaway Torque)*: The linear actuator minimum duty floor was too low to overcome internal gearbox static friction, causing buzzing without motion.
- *Theory B (Overshoot Limit Cycle)*: The duty floor was too high (40% and 70%), forcing even tiny corrections (0.2°) to kick at high power, overshooting the 1.5° deadband into the opposite direction.

Testing verified **Theory B**: setting minimum duty to 70% worsened the wobble, whereas setting `ACT_MIN_DUTY_PCT = 0.0f` completely eliminated the oscillation, allowing smooth PID convergence.

7.3 Case Study 3: 48V BLDC Commutation Noise on Encoders

Issue Description

Optical wheel encoders reported erratic spikes of $10,000 +$ RPM when BLDC motors were running under load.

Root Cause: High di/dt switching pulses from the 48V 3-phase BLDC inverter stages induced transient glitches on the high-impedance encoder lines.

Solution: Installed 100 nF (104) ceramic capacitors across Phase A/B to GND directly at the STM32 timer input pins (PA0/PA1, PA6/PA7), forming a hardware low-pass filter that completely eliminated spurious pulses.

7.4 Master Successor Quick-Troubleshooting Matrix

Table 7.1: Quick Troubleshooting Matrix for Common System Anomalies.

Symptom	Probable Cause	Immediate Corrective Action
Contactor clicks then opens immediately	RC transmitter off or SWD in DIS-ARM	Turn on FlySky transmitter, ensure SWD is UP and throttle is centered.
Steering motor buzzes but wheels do not move	Actuator linkage mechanically inbound or 12V fuse blown	Check 12V DCDC output fuse; inspect tie-rod ball joints for mechanical binding.
Rover drives straight but turns sluggishly	Kingpin pot slipped or zero-point offset drifted	Run ADC Calibration Runbook; adjust STEER_POT_CENTER_ADC in actuator.h.
Raspberry Pi cannot connect to STM32	Wrong UART device or permission issue	Verify connection on /dev/ttyAMA10; ensure user belongs to dialout group.

Chapter 8

Calibration Runbooks & Standard Operating Procedures

8.1 SOP 1: Kingpin Potentiometer ADC Calibration Runbook

To calibrate the absolute steering angle zero-point and full-throw limits:

1. Disconnect the PA-12 linear actuator mechanical clevis pin from the front steering tie-rod.
2. Manually align both front wheels to be perfectly parallel to the chassis center line using a straightedge.
3. Power on the STM32 and read raw ADS1115 ADC channel 0 counts over the debug serial port (DBG_TX).
4. Record the center value (nominally ≈ 16000 counts) and update STEER_POT_CENTER_ADC in STM/Rover_closed_loop/Core/Inc/actuator.h.
5. Manually steer to full left ($+45^\circ$) and full right (-45°), verifying counts span 3000 to 29000 counts linearly without saturation.
6. Re-attach the clevis pin and verify that commanded angles match measured physical wheel angles within $\pm 0.5^\circ$.

8.2 SOP 2: Safe Field Startup Sequence

Standard Field Power-On Sequence

1. Turn ON the FlySky FS-i6X 2.4GHz RC Transmitter. Verify SWD is in DISARM position (DOWN).
2. Turn ON the heavy-duty manual battery isolator switch.
3. Verify STM32 status LED (PC13) begins blinking at a steady 20 Hz heartbeat rate.
4. Listen for the 100A main contactor loud audible click engaging 48V power.
5. Flip SWD to ARM (UP). Flip SWA to SELECT mode (DOWN = Manual RC, UP = Auto RPi5).

Chapter 9

Strategic Engineering Roadmap for Successor

Based on the research document `For_future/ROVER_RESEARCH (1).txt`, the top strategic engineering priorities for the incoming engineer are:

9.1 Priority 1: Multi-Frequency RTK-GNSS + EKF Fusion

Dead-reckoning odometry drifts over extended multi-hectare operations. Integrate an on-board multi-frequency RTK GNSS receiver fused with optical wheel encoders and BNO055 IMU quaternions via an **Extended Kalman Filter (EKF)** running inside the companion autonomy stack.

9.2 Priority 2: Electronic Power Take-Off (e-PTO) Subsystem

Agricultural tools (rotary weeders, seeders, precision spray nozzles) require managed electrical power. Implement high-current solid-state relays controlled via STM32 GPIOs and companion software to automate implement engagement at row entry and disengagement at headlands.

9.3 Priority 3: Secondary Long-Range Hardware E-Stop

Because standard 2.4 GHz RC signals attenuate beyond 300 m in crop fields, deploy an independent 868/915 MHz LoRa or cellular hardware safety interlock that hard-breaks the contactor gate circuit upon watchdog expiry.